First R Session

01 Getting in: Posit Cloud

R is the engine that does the calculating. **RStudio** is the dashboard you drive it from. **Posit** is the company that makes RStudio. **Posit Cloud** is that same dashboard running inside a web browser — nothing to install, and every machine in the room identical.

You need three things before the session starts: a free account, the exercise file from GitHub, and the file uploaded into a project. Ten minutes, once.

1	Go to **posit.cloud** and click **Sign Up**. Choose the free **Cloud Free** plan. It does not expire and no card is asked for.
2	Download the exercise file for the day from github.com/statscourse26 — `day2-exercises.R` or `day3-exercises.R`. On the file's page click the **download raw file** button (the downward arrow, top right of the file). Do not copy and paste from the screen: the quotation marks change and the script stops working.
3	Back in Posit Cloud, click **New Project** → **New RStudio Project**. Wait a few seconds for four panes to appear. That is RStudio.
4	In the bottom right pane, open the **Files** tab and click **Upload** → **Choose File** → the `.R` file you just downloaded → **OK**. It appears in the file list.
5	Click its name in the list. The script opens in the top left pane, keeping the name it already has, and you are ready to work.

Two things that surprise people

A project falls asleep after about a quarter of an hour without activity — click back into it and it wakes where you left off, with nothing lost. And your files live in your Posit account, not on the classroom computer, so bring nothing but a browser. Make a second project for Day 3 rather than reusing the first: one project, one script, one afternoon.

If the upload does not work

Some hospital and practice networks block it. Then use **File → New File → R Script**, open the downloaded file in any text editor on your own machine, and copy the whole thing across in one go. Save it in the project with **Ctrl+S** (Mac: **Cmd+S**) under the same name.

02 The four panes

Whether you work in the browser or on your own machine, this is the whole geography of the program.

TOP LEFT

Editor (Source)

Where your script sits — the file you just uploaded. Nothing here runs until you tell it to. Put the cursor on a line and press **Ctrl+Enter** (Mac: **Cmd+Enter**) to send that line down to the console.

This is the pane that matters. Your script is your lab book.

TOP RIGHT

Environment

A list of everything you have made so far — every object, with its size and contents. If you cannot remember whether `depth` exists yet, look here.

Empty at the start of every session. That is normal and healthy.

BOTTOM LEFT

Console

Where commands actually run and answers appear. You can type here directly for quick throwaway things.

Nothing typed here is saved. Treat it as a scrap of paper, not as your work.

BOTTOM RIGHT

Files · Plots · Packages · Help

Four tabs sharing one pane. **Files** is where you uploaded the script; your graphs appear under **Plots**; documentation appears under **Help**.

If a plot looks wrong, the back arrow in Plots shows the previous one.

The one habit worth forming today

Write in the editor, run into the console. If you only ever type in the console, then tomorrow you will have the answers but not the questions — and you will not be able to repeat, correct, or publish anything you did.

THE PROMPTS YOU WILL SEE IN THE CONSOLE

`>`	R is ready and waiting for a command.
`+`	R is still waiting for you to finish — usually a missing bracket or quotation mark. Press **Esc** to escape and start again.
`Error: object 'x' not found`	You used something you have not made yet, or misspelled it. R is case-sensitive: `Depth` and `depth` are two different things.

Nothing in R is broken by a mistake. The worst that happens is an error message, and every error message is a sentence telling you what R could not do.

If you would rather work offline later

Install R from cran.r-project.org first, then RStudio Desktop from posit.co/downloads — in that order, because RStudio needs an engine to drive. Open the same `.R` file there. The four panes look and behave identically.

03 Your first commands

R is a very good calculator that never forgets. Type these in the editor, one line at a time, and send each one to the console.

```
2 + 2
[1] 4

12 / 5
[1] 2.4

sqrt(16)                # a function: a name, then round brackets
[1] 4
```

The `[1]` in front of the answer is not part of it. It means "this line of output starts with result number 1". You will stop noticing it within an hour.

STORING SOMETHING: THE ASSIGNMENT ARROW

Nothing is kept unless you give it a name. The arrow `<-` means put the thing on the right into the name on the left. Type it as a less-than sign followed by a minus, or press **Alt+–**.

```
depth <- 3.4            # make it. R prints nothing — that is success
depth                   # type the name to see it
[1] 3.4

depth * 2
[1] 6.8
```

The moment you run the first line, `depth` appears in the Environment pane. It stays there until you overwrite it or close R.

MORE THAN ONE NUMBER: C()

A single number is rarely useful. `c()` — for combine — glues values into a **vector**: R's word for a row of values of the same kind.

```
depth <- c(3.1, 2.8, 4.0, 3.6, 2.9, 3.4)
length(depth)           # how many values?
[1] 6

mean(depth)
[1] 3.3

depth - 1               # arithmetic applies to every value at once
[1] 2.1 1.8 3.0 2.6 1.9 2.4
```

This is R's best trick

You never write a loop to say "do this to each value". Write `depth * 2` and R does it to all six. This is called vectorisation, and almost every command in R behaves this way.

PICKING VALUES OUT: SQUARE BRACKETS

Round brackets call a function. Square brackets take a piece out of an object.

```
depth[1]                # the first value
[1] 3.1

depth[c(1, 3)]          # the first and the third
[1] 3.1 4.0

depth[-1]               # everything EXCEPT the first
[1] 2.8 4.0 3.6 2.9 3.4
```

The minus sign inside square brackets means "leave this out". It is not a subtraction and not a negative position — a small thing that confuses everyone exactly once.

04 Everything is an object

This is the idea the rest of R hangs on, and it is simpler than it sounds. Two sentences: everything that exists in R is an object, and everything that happens is a function. Your data are objects. A single number is an object. A plot is an object. Even a function is an object.

EVERY OBJECT KNOWS WHAT IT IS

An object does not only carry values. It also carries a label saying what kind of thing it is. Ask with `class()`.

```
depth   <- c(3.1, 2.8, 4.0)
group   <- c("A", "A", "B")
treated <- c(TRUE, TRUE, FALSE)

class(depth)     [1] "numeric"
class(group)     [1] "character"
class(treated)   [1] "logical"
```

`numeric`	Numbers: measurements, counts, ages.
`character`	Text, always in quotation marks: names, labels, groups.
`logical`	Only TRUE and FALSE. These come out of comparisons.
`factor`	A category with a fixed set of levels: sex, treatment arm, tooth type.

WHY THE LABEL MATTERS: ONE FUNCTION, MANY BEHAVIOURS

Here is the object-oriented part. When you call a function, R first asks the object what class it is, and then runs the version of the function written for that class. You use one name; R chooses the right machinery underneath.

```
summary(depth)                  # numeric -> minimum, quartiles, mean
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
   2.80    2.95    3.10    3.30    3.55    4.00

summary(factor(group))          # factor -> counts per level
A B
2 1
```

Same command, entirely different answers — because the object was different, not because you asked differently. The same is true of `print()` and `plot()`: plotting numbers gives a scatter of points, plotting a factor gives a bar chart. This is all that "object oriented" means in day-to-day R.

The practical consequence

When a command gives you a strange answer, the first question is never "is the command wrong?" It is `class(x)` — because nine times out of ten the object is not the kind of thing you thought it was. Numbers accidentally stored as text are the classic case.

LOOKING INSIDE ANYTHING: STR()

```
str(depth)      num [1:3] 3.1 2.8 4
str(group)      chr [1:3] "A" "A" "B"
```

`str()` is short for structure. It tells you the class, the size, and the first few values, all in one line. It is the single most useful command in R for finding out what you are actually holding.

05 Functions, tests, and missing values

HOW A FUNCTION IS BUILT

A function has a name, round brackets, and things you put inside them called **arguments**, separated by commas. Many arguments have sensible defaults, so you only name the ones you want to change.

```
round(3.14159)              # the default is 0 decimal places
[1] 3

round(3.14159, digits = 2)  # name the argument you want
[1] 3.14

round(3.14159, 2)           # same thing, by position
[1] 3.14
```

Naming arguments takes three more keystrokes and saves a great deal of confusion later. `?round` in the console opens the documentation in the Help pane, which lists every argument and its default.

ASKING QUESTIONS: COMPARISONS GIVE TRUE AND FALSE

```
depth <- c(3.1, 2.8, 4.0, 3.6, 2.9, 3.4)

depth > 3
[1]  TRUE FALSE  TRUE  TRUE FALSE  TRUE

depth[depth > 3]         # keep only the values that answered TRUE
[1] 3.1 4.0 3.6 3.4

sum(depth > 3)           # TRUE counts as 1, FALSE as 0
[1] 4
```

One equals sign or two?

= assigns or names an argument. == asks a question: is this equal to that? Writing one where you meant the other is the most common beginner error in every language, R included.

MISSING VALUES

A patient who did not turn up is NA, not zero and not blank. R refuses to guess: any calculation touching an NA returns NA, so that you notice.

```
depth <- c(3.1, 2.8, NA, 3.6)
mean(depth)
[1] NA

mean(depth, na.rm = TRUE)    # rm = remove
[1] 3.166667
```

Do not delete missing values from your data to keep R quiet. The NA is information: it records that something was not measured.

PACKAGES: BORROWING SOMEONE ELSE'S FUNCTIONS

Base R can do a great deal on its own, and both exercise scripts in this course use nothing else. Anything extra comes in a **package**: install it once, load it in every session that needs it.

```
install.packages("boot")     # once, ever — needs internet
library(boot)                # every session you want to use it
```

06 A whole small session

Everything on the previous four pages, in fifteen lines. Type it into a new script, save it as `first-session.R`, and run it line by line.

```
# --- probing depth after treatment, six patients ------------------
depth <- c(3.1, 2.8, 4.0, 3.6, 2.9, 3.4)
group <- factor(c("A", "A", "A", "B", "B", "B"))

str(depth)                   # what am I holding?
class(group)

mean(depth)                  # the whole sample
summary(depth)

depth[group == "A"]          # only the patients given A
mean(depth[group == "A"])
mean(depth[group == "B"])

tapply(depth, group, mean)   # both group means in one line

hist(depth)                  # a picture, in the Plots pane
boxplot(depth ~ group)       # one box per group
```

Read the last two lines again. `hist()` and `boxplot()` did not ask you what kind of data you had — they looked at the objects and decided. That is the whole lesson of section 04, doing real work.

FIVE HABITS THAT WILL SAVE YOU

- **Work in a script, not the console.** If it is worth running twice, it is worth saving.
- **Comment as you go.** Everything after a **#** is ignored by R and read by you in six months.
- **Name things for what they are.** `depth`, not `d`; `pilot`, not `data2`.
- **Restart R often** (Session → Restart R) and rerun your script from the top. If it still works, it really works.
- **Read the error out loud.** "Object not found" means you have not made it yet. "Could not find function" means a typo or a package you have not loaded.

Keeping your work

Posit Cloud saves the project by itself, and it will be waiting next week on any computer you log in from. To keep a copy of your own, use **Files → More → Export** in the bottom right pane, tick the files you want, and download them. Worth doing at the end of Day 3.

If you remember nothing else

Make objects with `<-`. Ask what they are with `class()` and `str()`. Do things to them with functions. That is R.

Where to go next, in this order: reading your own data in with `read.csv()`; the data frame, which is a table where each column is one of the vectors you already understand; and drawing better graphs. Everything after that is a variation on what is already on these five pages.